COMPUTING 2 - PARALLEL COMPUTING

Chapter 4 Mock Test

CUDA Indexing, Coalescing, and Synchronization

Suggested time	Total points	Coverage
55 minutes	60	1D/2D mapping, bounds, section indexing, barriers, race conditions, Unified Memory

NAME		STUDENT ID	
DATE		SCORE	/ 60

Instructions
- Answer every question and show essential reasoning for partial credit.
- For true/false items, an unsupported letter receives no credit.
- Unless stated otherwise, assume NVIDIA warp size 32 and ideal arithmetic.
- Keep the answer section closed until the suggested time has elapsed.

Questions

4.1 Two coalesced sections per block — 8 points

Each block processes two consecutive sections of blockDim.x elements. Every thread handles one element from the first section and then one from the second.

Write i for the first element, j for the second, and the complete bounds-checked vector-add body.

WORK AREA

4.2 Two-dimensional mapping — 7 points

For a row-major matrix with width W and height H, derive row, col, and linear index for a 2D grid and block. Include the bounds check.

WORK AREA

4.3 Launch geometry — 6 points

A vector has n=1,000,003 elements and each thread computes one element. With 256 threads/block, find the number of blocks, the number of launched threads, and the number of inactive threads in the last block.

WORK AREA

4.4 Transpose race — 12 points

The square tile size B is compile-time constant and ranges from 1 to 20. Determine for which B this kernel is guaranteed correct and repair it for all B.

```
__shared__ float tile[B][B];
int x = blockIdx.x * B + threadIdx.x;
int y = blockIdx.y * B + threadIdx.y;
tile[threadIdx.y][threadIdx.x] = A[y * width + x];
A[y * width + x] = tile[threadIdx.x][threadIdx.y];
```

WORK AREA

4.5 Grid-wide dependence — 9 points

Kernel phase 2 reads results written by neighboring blocks in phase 1. Explain why __syncthreads() is insufficient and give two correct implementation strategies.

WORK AREA

4.6 Coalesced adjacent-pair mapping — 8 points

Each thread computes two adjacent outputs. Compare the first-element mappings below for correctness and coalescing, then choose the preferred one.

```
(a) i = 2 * blockIdx.x * blockDim.x + threadIdx.x;
(b) i = 2 * (blockIdx.x * blockDim.x + threadIdx.x);
```

WORK AREA

4.7 Unified Memory reasoning — 10 points

A program replaces cudaMalloc/cudaMemcpy with cudaMallocManaged and becomes slower on first use of a large array. Explain correctness versus performance and name two remedies.

WORK AREA

ANSWER SECTION

Chapter 4: Answer Key & Worked Solutions

Grade the method as well as the final result. Numerical answers show enough intermediate work to identify where an error occurred.

Quick Answer Key

Q	Topic	Short answer
4.1	Two coalesced sections per block	i=2*blockIdx.x*blockDim.x+threadIdx.x; j=i+blockDim.x
4.2	Two-dimensional mapping	row=by*BDy+ty; col=bx*BDx+tx; idx=row*W+col
4.3	Launch geometry	3907 blocks; 1,000,192 threads; 189 inactive
4.4	Transpose race	Under the course's 32-thread warp model, B=1..5; add __syncthreads() for all B=1..20
4.5	Grid-wide dependence	Barrier is block-local; split phases into kernels or use a supported cooperative grid sync
4.6	Coalesced adjacent-pair mapping	Both can cover pairs with suitable second index, but (a) gives coalesced first accesses; use second i+blockDim.x
4.7	Unified Memory reasoning	Correct unified addressability; slower from page faults/migration. Prefetch, advise placement, or initialize near use.

Worked Solutions

4.1 Two coalesced sections per block — 8 points

Answer. Threads in a warp should access adjacent first elements together, then adjacent second elements together.

$$i = 2 \times blockIdx.x \times blockDim.x + threadIdx.x$$

$$j = i + blockDim.x$$

```
int i = 2 * blockIdx.x * blockDim.x + threadIdx.x;
int j = i + blockDim.x;
if (i < n) C[i] = A[i] + B[i];
if (j < n) C[j] = A[j] + B[j];
```

4.2 Two-dimensional mapping — 7 points

Answer. The x dimension maps naturally to columns and the y dimension to rows.

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;
if (row < H && col < W) {
    int idx = row * W + col;
    // use A[idx]
}
```

4.3 Launch geometry — 6 points

Answer. Use integer ceiling division.

- blocks=(n+255)/256=floor(1,000,258/256)=3907.
- launched threads=3907x256=1,000,192.
- inactive threads=1,000,192-1,000,003=189.

$$gridDim.x = (n + blockDim.x - 1) / blockDim.x$$

4.4 Transpose race — 12 points

Answer. Under the intended lecture model, the kernel works when the entire B by B block fits in one 32-thread warp. Once the block spans multiple warps, one warp may read values before another warp has written them.

- Require $B^2 \le 32$. For integer B in 1..20, this gives B=1,2,3,4,5.
- For B=6..20, the block has multiple warps and the cross-warp producer-consumer race is possible.
- A block barrier after the shared-memory write makes all tile values visible before transposed reads and supports every B in the stated range.

```
tile[threadIdx.y][threadIdx.x] = A[y * width + x];
__syncthreads();
A[y * width + x] = tile[threadIdx.x][threadIdx.y];
```

Common trap. For portable general code, use the barrier even if a one-warp case appears to work. Do not place __syncthreads() inside a branch that only some threads execute.

4.5 Grid-wide dependence — 9 points

Answer. CUDA normally schedules blocks independently and does not guarantee that all blocks are resident together.

- Strategy 1: end kernel 1 and launch kernel 2; the launch boundary provides global ordering in the same stream.
- Strategy 2: use cooperative groups grid synchronization only with a cooperative launch and its residency constraints.
- Atomics can protect individual updates but do not automatically create a whole-grid phase barrier.

Common trap. A spin barrier in global memory can deadlock when unscheduled blocks are needed to release resident blocks.

4.6 Coalesced adjacent-pair mapping — 8 points

Answer. Option (a) assigns adjacent first-section elements to adjacent lanes. Option (b) makes a warp access every other element on its first instruction.

- For (a), lane t reads base+t: unit stride and coalesced.
- Its second element should be base+blockDim.x+t, not i+1.
- For (b), lane t reads base+2t: stride 2 and reduced transaction utilization, even though each thread's pair i and i+1 is adjacent.

Common trap. Coalescing is judged across lanes for one instruction, not across two sequential instructions issued by one lane.

4.7 Unified Memory reasoning — 10 points

Answer. Unified Memory gives a common virtual address and managed migration. It does not promise that pages begin in the best physical location.

- First GPU touch can trigger page faults and CPU-to-GPU page migration.
- Use cudaMemPrefetchAsync before the kernel when the destination device is known.
- Use memory advice, appropriate initialization/first-touch, or explicit copies when predictable movement is preferable.